Decorator

Season: 2 · Structural · Category: GoF · Level: Beginner

Intent

- Attach responsibilities dynamically by wrapping.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Stacking phone cases: waterproof + battery.

When to Use

- See Intent.
- The design force named in Intent shows up more than once.

When Not to Use

- When a simpler redesign removes the need.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.gof.structural.decorator;

interface DataSource {
    String read();
    void write(String data);
}

final class InMemoryDataSource implements DataSource {
    private String data = "";
    public String read() { return data; }
    public void write(String data) { this.data = data; }
}

abstract class DataSourceDecorator implements DataSource {
    protected final DataSource wrappee;
    protected DataSourceDecorator(DataSource wrappee) { this.wrappee = wrappee; }
    public String read() { return wrappee.read(); }
    public void write(String data) { wrappee.write(data); }
}

final class EncryptionDecorator extends DataSourceDecorator {
    EncryptionDecorator(DataSource wrappee) { super(wrappee); }

    @Override public void write(String data) {
        super.write(new StringBuilder(data).reverse().toString());
    }

    @Override public String read() {
        return new StringBuilder(super.read()).reverse().toString();
    }
```

Classic Implementation (2)

```
}

/** Stackable decorator demo entry — compression over encryption over memory. */
public final class DataSourceDecorators {
    private DataSourceDecorators() {}

    public static DataSource encryptedCompressedMemory() {
        return new EncryptionDecorator(new CompressionDecorator(new InMemoryDataSource()));
    }
}

final class CompressionDecorator extends DataSourceDecorator {
    CompressionDecorator(DataSource wrappee) { super(wrappee); }

    @Override public void write(String data) {
        super.write("c:" + data.replace(" ", ""));
    }

    @Override public String read() {
        var s = super.read();
        return s.startsWith("c:") ? s.substring(2) : s;
    }
}
```

Modern Java 25 / Functional

- Prefer sealed interfaces + records; virtual proxy uses synchronized lazy init.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: gof/structural/decorator/
```

Pitfalls

- Keep adapters thin.
- Decorators should preserve the interface.
- Facades must not become god objects.

Interview Q&A

- Q: When is Decorator the wrong choice? A: When you can change the adaptee/subject API or collapse the hierarchy.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Attach responsibilities dynamically by wrapping.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Stacking phone cases: waterproof + battery. [0:40] Intent — State the design force in one sentence: ...